

Chapter 8

Core Java: Arrays, Strings, StringBuffer/StringBuilder & Threads

1. What Is an Array?

An array is a collection of elements of the same data type stored in contiguous memory locations. It allows you to store multiple values of the same type in a single variable.

Without an Array (Repetitive)

```
int mark1 = 85;  
int mark2 = 90;  
int mark3 = 75;  
int mark4 = 95;  
int mark5 = 88;
```

Hard to manage and repetitive!

With an Array (Clean)

```
int[] marks = {85, 90, 75, 95, 88};
```

Easy to manage, loop, and process.

Key Points

- Array index starts at 0.
- Array size is fixed once created.
- Array elements must all be of the same type.

```
dataType[] arrayName = new dataType[size];
```

// Example:

```
int[] numbers = new int[5];  
numbers[0] = 10;  
numbers[1] = 20;  
numbers[2] = 30;  
numbers[3] = 40;  
numbers[4] = 50;
```

Advantages of Arrays

Advantage	Description
Code Simplicity	Store multiple values in one variable.
Easy Access	Access any element using its index.
Iteration	Easy to process using loops.
Memory Efficiency	Stored in continuous memory.
Sorting/Searching	Easy to apply algorithms.

Example — Storing and Printing Array Elements

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40, 50};  
        System.out.println("Array elements are:");  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.println(numbers[i]);  
        }  
    }  
}
```

Array elements are:

10
20
30
40
50

2. Types of Arrays

2.1 One-Dimensional (1D) Array

A single row of elements — a list of elements of the same data type stored in a linear form.

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {5, 10, 15, 20};  
        int sum = 0;  
        for (int i = 0; i < arr.length; i++) {  
            sum += arr[i];  
        }  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output: Sum = 50. Access example: `int[] arr = {1,2,3,4,5}; arr[2]` gives 3.

2.2 Two-Dimensional (2D) Array / Matrix

An array of arrays, used to represent tables — data in rows and columns.

```
public class Main {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
    }  
}
```

```

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
1 2 3
4 5 6
7 8 9

```

Access example: matrix[1][2] gives 6.

Difference Between 1D and 2D Arrays

Feature	1D Array	2D Array
Definition	Stores data in a single row (linear) form	Stores data in rows and columns (table) form
Syntax	dataType[] array = new dataType[size];	dataType[][] array = new dataType[rows][cols];
Structure	Single line of elements	Matrix-like structure
Access	array[index]	array[row][column]
Memory Representation	Continuous block of elements	Continuous blocks of sub-arrays
Example	int[] marks = {85, 90, 75};	int[][] matrix = {{1,2}, {3,4}};
Use Case	Storing a list of items (e.g. student marks)	Representing grids, tables, or matrices
Size	Only one dimension (length)	Two dimensions (rows x columns)
Loops Required	One loop (for)	Nested loops (for inside for)

Choosing 1D vs 2D — Example Scenarios

Scenario	Recommended Array	Example
Storing 10 student marks	1D Array	int[] marks = new int[10];
Representing a 3x3 matrix	2D Array	int[][] matrix = new int[3][3];
Student names list	1D Array	String[] names = {"Asha", "Ravi", "Meena"};
Student marks per subject	2D Array	int[][] marks = {{85,90}, {78,88}, {92,95}};

Combined Example — 1D and 2D Together

```

public class ArrayComparison {
    public static void main(String[] args) {
        // 1D Array
        int[] scores = {10, 20, 30, 40, 50};
    }
}

```

```

        System.out.println("1D Array:");
        for (int i = 0; i < scores.length; i++) {
            System.out.print(scores[i] + " ");
        }

        // 2D Array
        int[][] table = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        System.out.println("\n\n2D Array:");
        for (int i = 0; i < table.length; i++) {
            for (int j = 0; j < table[i].length; j++) {
                System.out.print(table[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

1D Array:
10 20 30 40 50

2D Array:
1 2 3
4 5 6
7 8 9

1D vs 2D Array — Quick Summary

Feature	1D Array	2D Array
Shape	Line	Table
Access	Single index	Two indices
Memory	Linear	Tabular
Loops	Single	Nested
Example use	Marks of students	Marks of students in multiple subjects

Practice Questions — 1D & 2D Arrays

- Write a program to store and print 5 student marks using a 1D array.
- Create a 2D array to store and display a 3x3 matrix.
- Find the sum of all elements in a 1D array.
- Write a program to add two 2D matrices and display the result.
- Display the diagonal elements of a 2D array.
- Exercise: Find the largest number in an array.

- Exercise: Find the sum of all elements.
- Exercise: Add two matrices.
- Exercise: Print the array in reverse order.
- Exercise: Find the second largest number.
- Exercise: Search for an element in the array.
- Exercise: Sort the array in ascending order.
- Exercise: Multiply two matrices.
- Exercise: Input a 2D array and print it in matrix form.
- Exercise: Print the transpose of a matrix (row becomes column).

3. Jagged Array in Java

A Jagged Array is a two-dimensional array where each row can have a different number of columns. In a normal 2D array, all rows have the same length; in a jagged array, each row can have a different length. It is also called an "array of arrays".

Visualization

```
[
  [10, 20],
  [30, 40, 50],
  [60]
]
// Row 0 -> 10, 20
// Row 1 -> 30, 40, 50
// Row 2 -> 60
```

Each row has a different size — this is the defining feature of a jagged array.

Jagged Array Syntax

Step 1: Declare a 2D array with a fixed number of rows, but leave the columns unspecified.

```
dataType[][] arrayName = new dataType[numberOfRows][];
```

Step 2: Create each row separately, with its own column size.

```
arrayName[0] = new dataType[size1];
arrayName[1] = new dataType[size2];
arrayName[2] = new dataType[size3];
```

```
// Full syntax example:
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[1];
```

Why Jagged Arrays Are Useful (Advantages)

Advantage	Explanation
Saves memory	No unused extra columns like in regular 2D arrays.

Flexible structure	Each row can be a different size.
Better representation	Useful when data is naturally uneven.
Faster access	Accessing specific rows is quick.

Real-Life Uses of Jagged Arrays

- Seats in a cinema hall (different rows have different seat counts).
- Days in months (months have 28, 30, or 31 days).
- Student marks (each student may have a different number of subjects).
- Playlists (each playlist has a different number of songs).

Example 1 — Simple Jagged Array (Initial Values)

```
public class JaggedArrayExample {
    public static void main(String[] args) {
        int[][] jagged = {
            {10, 20},
            {30, 40, 50},
            {60}
        };

        System.out.println("Jagged Array Elements:");
        for (int i = 0; i < jagged.length; i++) {
            for (int j = 0; j < jagged[i].length; j++) {
                System.out.print(jagged[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Jagged Array Elements:
10 20
30 40 50
60

Example — Jagged Array Assigned Row by Row

```
public class Main {
    public static void main(String[] args) {
        int[][] jagged = new int[3][];
        jagged[0] = new int[]{1, 2, 3};
        jagged[1] = new int[]{4, 5};
        jagged[2] = new int[]{6, 7, 8, 9};

        for (int i = 0; i < jagged.length; i++) {
            for (int j = 0; j < jagged[i].length; j++) {
                System.out.print(jagged[i][j] + " ");
            }
        }
    }
}
```

```

        }
        System.out.println();
    }
}
1 2 3
4 5
6 7 8 9

```

Example 2 — Creating a Jagged Array Using Loops (User Input)

```

import java.util.Scanner;

public class JaggedArrayInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[][] arr = new int[3][];
        // Defining different sizes for each row
        arr[0] = new int[2];
        arr[1] = new int[4];
        arr[2] = new int[1];

        System.out.println("Enter elements for jagged array:");
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        System.out.println("Jagged Array Output:");
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

Example 3 — Sum of Each Row in a Jagged Array

```

public class JaggedArraySum {
    public static void main(String[] args) {
        int[][] arr = {
            {1, 2},
            {3, 4, 5},

```

```

        {6}
    };

    for (int i = 0; i < arr.length; i++) {
        int sum = 0;
        for (int j = 0; j < arr[i].length; j++) {
            sum += arr[i][j];
        }
        System.out.println("Sum of row " + i + ": " + sum);
    }
}

```

Sum of row 0: 3
Sum of row 1: 12
Sum of row 2: 6

Difference Between a Regular 2D Array and a Jagged Array

Feature	Normal 2D Array	Jagged Array
Shape	Rectangle	Irregular
Row size	Same for all rows	Different sizes allowed
Memory	Can waste memory	Memory efficient
Structure	Matrix	Array of arrays
Example	3x3 table	{2 elements, 3 elements, 1 element}

Practice Questions — Jagged Arrays

- Create a jagged array of 3 rows: Row1 - 2 elements, Row2 - 4 elements, Row3 - 3 elements.
- Store student marks where each student has a different number of subjects.
- Take input for a jagged array and print the largest value in each row.
- Print the jagged array in reverse order.
- Find the total number of elements in a jagged array.

4. Strings in Java

A String is a sequence of characters enclosed within double quotes, e.g. String name = "Java";. In Java, String is a class, not a primitive data type.

Strings Are Immutable

Once created, a string cannot be changed — any modification creates a brand-new String object instead of altering the original.

```

public class Main {
    public static void main(String[] args) {
        String s1 = "Hello";
        s1 = s1 + " World"; // creates a NEW String object
        System.out.println(s1);
    }
}

```

```
}  
}
```

Output: Hello World (the original "Hello" object still exists in memory but is no longer referenced by s1).

Important Methods of the String Class

```
// length() - returns the number of characters  
System.out.println("Hello".length()); // 5  
  
// charAt() - returns the character at an index  
char c = "Java".charAt(2); // 'v'  
  
// substring() - extracts part of a string  
String s = "Programming";  
System.out.println(s.substring(0, 5)); // Progr  
  
// equals() / equalsIgnoreCase()  
"java".equals("Java");           // false (case-sensitive)  
"java".equalsIgnoreCase("Java"); // true  
  
// toUpperCase() / toLowerCase()  
"hello".toUpperCase(); // HELLO  
  
// replace()  
"apple".replace('p', 'b'); // "abble"  
  
// split()  
String[] words = "I love Java".split(" ");  
  
// trim() - removes leading/trailing spaces  
"hello ".trim(); // "hello"
```

Example — String Operations Together

```
public class Main {  
    public static void main(String[] args) {  
        String name = "Java Programming";  
  
        System.out.println("Length: " + name.length());  
        System.out.println("Uppercase: " + name.toUpperCase());  
        System.out.println("Lowercase: " + name.toLowerCase());  
        System.out.println("Character at index 5: " + name.charAt(5));  
        System.out.println("Contains 'Pro': " + name.contains("Pro"));  
    }  
}
```

Length: 16

Uppercase: JAVA PROGRAMMING

Lowercase: java programming
Character at index 5: P
Contains 'Pro': true

String Method Quick Reference

Method	Description	Example	Output
length()	Returns length	"Hello".length()	5
charAt(i)	Returns char at index	"Java".charAt(2)	v
toUpperCase()	Converts to upper case	"java".toUpperCase()	JAVA
toLowerCase()	Converts to lower case	"JAVA".toLowerCase()	java
equals()	Compares two strings	"Java".equals("JAVA")	false
concat()	Joins two strings	"Hello".concat("World")	HelloWorld

5. StringBuffer & StringBuilder

StringBuffer and StringBuilder are used to create and modify strings efficiently. Normally, String is immutable, so every modification creates a new object. To avoid this overhead, Java provides these two mutable alternatives.

When to Use Which

- Use StringBuffer — when working in a multi-threaded environment.
- Use StringBuilder — when working in a single-threaded environment (most common case).

Memory Trick

- Buffer = Safe (thread-safe).
- Builder = Fast (no safety overhead).

Real-Life Analogy

- StringBuffer -> a bank locker (safe, controlled access).
- StringBuilder -> a notebook (fast writing, no restriction).

StringBuffer — Definition

StringBuffer is a predefined class used to create mutable (modifiable) strings — the content can change after creation without creating a new object. It is also thread-safe, because all its methods are synchronized.

Key Points — StringBuffer

- Introduced in Java 1.0.
- Mutable string (can be changed).
- Thread-safe (synchronized).
- Slower than StringBuilder.
- Stored in the java.lang package.

Why StringBuffer is used: in String, every change creates a new object, wasting memory. StringBuffer modifies the same object instead — efficient.

Example 1 — append(), insert(), replace(), reverse()

```
public class Main {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Java");  
  
        sb.append(" Programming");           // add text at end -> "Java  
Programming"  
        sb.insert(4, " Language");           // insert at position 4 ->  
"Java Language Programming"  
        sb.replace(0, 4, "Core");           // replace part -> "Core  
Language Programming"  
        sb.reverse();                       // reverse the whole string  
  
        System.out.println(sb);  
    }  
}
```

Output: gnimmargorP egaugnaL eroC (the fully reversed result of "Core Language Programming").

Example 2 — Real-Life Usage

```
public class Main {  
    public static void main(String[] args) {  
        StringBuffer message = new StringBuffer("Hello");  
        message.append(" Student");  
        message.append(", Welcome to Java Class");  
        System.out.println(message);  
    }  
}
```

Output: Hello Student, Welcome to Java Class

StringBuilder — Definition

StringBuilder is a class used to create mutable strings, just like StringBuffer, but it is NOT synchronized, so it is faster.

Key Points — StringBuilder

- Introduced in Java 1.5.
- Mutable string.
- Not thread-safe.
- Faster than StringBuffer.
- Best for single-threaded programs.

Why StringBuilder is used: when performance is important and no multi-threading is needed.

Example 1 — append(), delete(), replace()

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Welcome");

        sb.append(" to Java");    // "Welcome to Java"
        sb.delete(0, 3);          // remove characters 0-2 ("Wel") -
> "come to Java"
        sb.replace(0, 5, "Hi");   // replace characters 0-4 ("come
") with "Hi"

        System.out.println(sb);
    }
}
```

Output: Hito Java (replace(0, 5, "Hi") removes "come " — including the trailing space — and inserts "Hi" with no space before "to", so the words run together).

Example 2 — Reverse a String

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Computer");
        System.out.println("Original: " + sb);
        System.out.println("Reverse: " + sb.reverse());
    }
}
```

Original: Computer

Reverse: retupmoC

Summary Table — String vs StringBuffer vs StringBuilder

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable, so inherently safe)	Yes	No
Speed	Slow	Medium	Fast
Suitable for	Normal text	Multithreading	Single-thread

Practice Questions — Strings

- Write a program to count vowels in a string.
- Reverse a string using charAt().
- Compare two strings using equals().
- Replace all spaces in a string with -.
- Convert the string "java programming" to uppercase.
- Extract "gram" from "Programming".
- Check if a string starts with "Hello".

- Count the words in "Java is fun".
- Remove extra spaces using trim().
- Use StringBuilder to reverse a sentence.

6. What Is a Thread?

A thread is the smallest unit of a program that can run independently. Threads allow a program to do multiple tasks at the same time — like doing multitasking in real life.

Real-Life Example

- You are listening to music (one thread).
- You are downloading a file (another thread).
- You are browsing the web (a third thread).

All happening at the same time -> Multithreading.

Example (Java)

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}
```

Important Thread Methods

Method	Description
start()	Starts a new thread.
run()	Defines the thread's work.
sleep(ms)	Pauses the thread.
join()	Waits for another thread to finish.
getName()	Gets the thread's name.
setPriority()	Sets the thread's priority.

7. What Is synchronized?

synchronized means only one thread is allowed to use a method or block of code at a time. It is like a door lock! If one thread enters, the door is locked, and other threads must wait.

Synchronized Method Example

```
synchronized void print() {
    System.out.println("Hello");
}
```

Purpose

- Protects shared data.
- Prevents two threads from modifying the same variable at the same time.

8. Tips for This Chapter

1. For arrays, always check the boundary condition first: index starts at 0, and the last valid index is `length - 1`, not `length`. Using `arr[arr.length]` is the single most common cause of an `ArrayIndexOutOfBoundsException`.
2. Think of a 2D array as 'a 1D array where each element is itself an array'. That's exactly why you need nested loops — the outer loop walks through the rows (the outer array), and the inner loop walks through each row's own elements.
3. The single question that tells a jagged array apart from a regular 2D array: 'do all rows have the same length?' If yes, it's a regular (rectangular) 2D array. If rows can differ, it's jagged — and you must declare it as `new dataType[rows][]` (no column size) and assign each row separately.
4. When looping through a jagged array, never hardcode the column count — always use `arr[i].length` inside the inner loop (not a fixed number), since each row can have a different length.
5. Remember Strings are immutable: operations like `toUpperCase()`, `replace()`, `substring()`, and `concat()` don't change the original string — they return a brand NEW string. If you don't assign the result back to a variable (e.g., `name.toUpperCase()`; with no assignment), the result is simply discarded.
6. Use `equals()` to compare String content, never `==`. The `==` operator compares references (memory addresses), not the actual characters — two strings with identical content can still fail an `==` check depending on how they were created.
7. Choose `StringBuilder` by default for building/modifying strings in ordinary (single-threaded) programs — it's faster. Only reach for `StringBuffer` specifically when multiple threads might modify the same string object at the same time.
8. For `StringBuffer/StringBuilder` methods that take index ranges (like `replace(start, end, str)` or `delete(start, end)`), remember the end index is EXCLUSIVE — `replace(0, 5, ...)` affects indices 0 through 4, not 0 through 5. Miscounting this is a common source of off-by-one bugs, as seen in the `StringBuilder 'Hito Java'` example.
9. Threads and `synchronized` are introduced only briefly here — the core idea to hold onto is: threads let tasks run concurrently, and `synchronized` is the tool that prevents two threads from corrupting shared data by touching it at the exact same time.